

PRÁCTICA CALIFICADA N.º 02 – SEMANA 3

Curso: Programación Orientada a Objetos II

Unidad Didáctica: I – Algoritmos y Diseño

Semana: 3

Modalidad: Presencial/Laboratorio

Docente: Mg. Aldo Hernán Zanabria Galvez

Duración: 2 horas

I. OBJETIVOS

- Aplicar el uso de objetos constantes y funciones constantes.
- Implementar composición de clases (objetos como miembros).
- Utilizar funciones friend y el puntero this en C++.
- Comparar los conceptos de encapsulamiento y visibilidad entre C++ y Python.
- Analizar cómo se implementan conceptos equivalentes en Python.

II. CONTENIDO DE LA PRÁCTICA

Se trabajará sobre los siguientes conceptos clave:

1. Objetos y funciones const.
2. Composición (clases que contienen otras clases).
3. Funciones amigas (friend).
4. El puntero this y llamadas encadenadas (return *this).
5. Creación dinámica de objetos (new y delete).
6. Comparativa estructurada entre **C++** y **Python**.

III. ACTIVIDADES

Actividad 3.1 – Constantes y Funciones Constantes

Enunciado:

Utilizando la clase Time vista en la práctica anterior, declare un objeto constante const Time, e intente utilizar métodos set y get para analizar qué funciones requieren ser const.

Instrucciones:

- En C++: Implementa getHour() const, printMilitary() const, etc.
- En Python: Simula el concepto de inmutabilidad usando propiedades @property.

Comparar comportamiento.

Actividad 3.2 – Composición de Clases

Enunciado:

Crea una clase Empleado que contenga dos objetos tipo Fecha:

- FechaNacimiento

- FechaContratacion

Instrucciones:

- Implementa ambas clases (Empleado y Fecha) en C++ y Python.
- Asegúrate de inicializar los objetos miembro correctamente en el constructor.

Comparar el orden de creación y destrucción de objetos.

Actividad 3.3 – Uso de funciones friend

Enunciado:

Declara una función friend void modificarX(Contador &, int) que modifique una variable privada x de la clase Contador.

Instrucciones:

- En C++, utiliza la palabra clave friend.
- En Python, permite acceso directo (Python no tiene friend, pero se puede simular controlando acceso).

Comparar control de acceso y seguridad.

Actividad 3.4 – El puntero this y llamadas encadenadas

Enunciado:

Implementa una clase Tiempo que permita llamadas encadenadas como:

```
t.setHora(10).setMinuto(30).setSegundo(15);
```

Instrucciones:

- En C++, retorna *this en cada método set.
- En Python, retorna self.

Comparar el encadenamiento de métodos.

Actividad 3.5 – Creación dinámica de objetos

Enunciado:

Crea un objeto dinámicamente de la clase Empleado usando new y elimínalo con delete.

Instrucciones:

- En C++: Utilizar punteros.
- En Python: No es necesario usar new; el recolector de basura gestiona los objetos.

Comparar gestión de memoria.

IV. ANÁLISIS COMPARATIVO

Completa la siguiente tabla para al menos **tres de las actividades anteriores**:

Concepto	Implementación en C++	Implementación en Python	Análisis Comparativo
Función const	int getHora() const;	@property	Python no impone restricciones como const
Composición de objetos	Fecha nacimiento; dentro de Empleado	self.nacimiento = Fecha()	Python ejecuta primero el constructor del objeto
Función friend	friend void modificarX(Contador&, int);	No existe; se simula con acceso directo	C++ otorga acceso explícito

V. PRODUCTO FINAL

El estudiante entregará:

- Implementación de las actividades en **C++ y Python**.
- Capturas de pantalla de la ejecución.
- Tabla comparativa completada.
- Breve reflexión individual (1/2 página): ¿qué ventajas encuentras al comparar ambos lenguajes desde la orientación a objetos?

ESTADO DEL ARTE: CLASES, ABSTRACCIÓN Y COMPOSICIÓN EN LA PROGRAMACIÓN ORIENTADA A OBJETOS (C++)

1. Introducción

La programación orientada a objetos (POO) constituye un paradigma esencial en el diseño y desarrollo de software moderno. A través de la encapsulación, la herencia, la abstracción y el polimorfismo, permite representar entidades del mundo real como objetos de software. Dentro de este enfoque, el lenguaje C++ ofrece herramientas potentes para definir y manipular clases, estructuras de datos complejas y modelos de comportamiento robustos. Este estado del arte se centra en los conceptos avanzados abordados en los capítulos 6 y 7 del texto base, relacionados con clases, estructuras, funciones friend, composición, constructores, destructores y el puntero `this`.

2. Estructuras y Abstracción de Datos

En los primeros pasos hacia la abstracción en C++, se utiliza la definición de estructuras (`struct`) para agrupar variables relacionadas. Un ejemplo clásico es la estructura `Time`, que agrupa tres miembros: hora, minuto y segundo.

```
struct Time {  
    int hour;  
    int minute;  
    int second;  
};
```

Este tipo de definición permite crear variables de tipo `Time` y manipularlas directamente mediante el operador punto (`.`). No obstante, el uso de `struct` en C++ es limitado en cuanto a encapsulamiento, ya que sus miembros son públicos por defecto. La evolución natural de esta forma de abstracción es el uso de clases.

3. Clases y Encapsulamiento

Una clase es una unidad de programación que permite encapsular atributos (datos) y operaciones (métodos). En contraste con las estructuras, los miembros de una clase son privados por defecto, lo que permite ocultar la implementación interna del objeto y controlar el acceso mediante funciones públicas.

```
class Time {  
public:  
    Time(); // Constructor  
    void setTime(int, int, int);  
    void printMilitary() const;  
    void printStandard() const;  
  
private:  
    int hour;  
    int minute;  
    int second;
```

```
};
```

El uso de funciones públicas como setTime() garantiza que los valores internos sean válidos, aplicando verificaciones de rango dentro de su implementación:

```
void Time::setTime(int h, int m, int s) {  
    hour = (h >= 0 && h < 24) ? h : 0;  
    minute = (m >= 0 && m < 60) ? m : 0;  
    second = (s >= 0 && s < 60) ? s : 0;  
}
```

Estas técnicas refuerzan el concepto de **información oculta** y permiten que el objeto mantenga un estado consistente.

4. Separación de Interfaz e Implementación

Una buena práctica en el desarrollo en C++ consiste en separar la **declaración de clases** en archivos de cabecera (.h) y la **implementación** de sus funciones en archivos fuente (.cpp). Esto favorece el mantenimiento y la reutilización del código.

Ejemplo de archivo time1.h:

```
#ifndef TIME1_H  
#define TIME1_H  
  
class Time {  
public:  
    Time();  
    void setTime(int, int, int);  
    void printMilitary() const;  
    void printStandard() const;  
  
private:  
    int hour;  
    int minute;  
    int second;  
};  
  
#endif
```

Y su implementación en time1.cpp:

```
#include "time1.h"  
#include <iostream>  
using std::cout;  
  
Time::Time() : hour(0), minute(0), second(0) {}  
  
void Time::printMilitary() const {  
    cout << (hour < 10 ? "0" : "") << hour << ":"  
        << (minute < 10 ? "0" : "") << minute;  
}
```

```

void Time::printStandard() const {
    cout << ((hour == 0 || hour == 12) ? 12 : hour % 12)
        << ":" << (minute < 10 ? "0" : "") << minute
        << ":" << (second < 10 ? "0" : "") << second
        << (hour < 12 ? " AM" : " PM");
}

```

5. Composición de Clases

La **composición** permite que una clase contenga objetos de otras clases como atributos. Es una técnica fundamental en la modelación del mundo real, ya que permite construir clases más complejas a partir de otras más simples.

Ejemplo:

```

class Fecha {
public:
    Fecha(int d, int m, int a);
    void mostrarFecha() const;

private:
    int dia;
    int mes;
    int anio;
};

class Empleado {
public:
    Empleado(const std::string&, const Fecha&, const Fecha&);
    void mostrarEmpleado() const;

private:
    std::string nombre;
    Fecha fechaNacimiento;
    Fecha fechaContratacion;
};

```

Aquí, la clase Empleado está compuesta por dos objetos de tipo Fecha. Este patrón de diseño permite reutilizar clases sin duplicar código, lo cual mejora la mantenibilidad y la legibilidad.

6. Funciones Amigas (friend)

Las funciones friend permiten el acceso a miembros privados de una clase desde funciones externas o desde otras clases. Aunque su uso debe ser moderado para no violar el encapsulamiento, son útiles en situaciones donde dos clases necesitan interactuar estrechamente.

Ejemplo:

```

class Contador {
    friend void modificarX(Contador&, int);
public:

```

```

    Contador() : x(0) {}
    void mostrar() const { std::cout << x << std::endl; }
private:
    int x;
};

void modificarX(Contador& c, int nuevoValor) {
    c.x = nuevoValor;
}

```

7. El puntero `this` y llamadas encadenadas

El puntero `this` es un puntero implícito que apunta al objeto actual. Se utiliza, por ejemplo, para retornar una referencia al objeto en métodos modificadores, permitiendo así el **encadenamiento de llamadas**.

```

class Tiempo {
public:
    Tiempo& setHora(int h) {
        hora = h; return *this;
    }

    Tiempo& setMinuto(int m) {
        minuto = m; return *this;
    }

    Tiempo& setSegundo(int s) {
        segundo = s; return *this;
    }

    void mostrar() const {
        std::cout << hora << ":" << minuto << ":" << segundo << std::endl;
    }

private:
    int hora, minuto, segundo;
};

// Uso:
Tiempo t;
t.setHora(12).setMinuto(30).setSegundo(15).mostrar();

```

8. Miembros Estáticos

Los miembros `static` pertenecen a la clase y no a instancias individuales. Se utilizan comúnmente para llevar la cuenta de cuántos objetos se han creado, o para definir atributos comunes a todas las instancias.

```

class ContadorObjetos {
public:
    ContadorObjetos() { ++cantidad; }
    static int obtenerCantidad() { return cantidad; }
}

```

```
private:
    static int cantidad;
};

int ContadorObjetos::cantidad = 0;
```

9. Constructores y Destructores

El constructor inicializa los miembros de un objeto, mientras que el destructor libera recursos cuando el objeto deja de existir. En C++, los destructores son especialmente importantes cuando se maneja memoria dinámica.

```
class Archivo {
public:
    Archivo() { archivo = fopen("datos.txt", "w"); }
    ~Archivo() { fclose(archivo); }

private:
    FILE* archivo;
};
```

10. Comparación con Python

En Python, la sintaxis es más simple y flexible. Aunque no existe friend, el encapsulamiento se basa en convenciones como el uso de `_atributo` o `__atributo`. La composición, el encadenamiento de métodos, la reutilización y los constructores (`__init__`) están presentes con distintas reglas de acceso y gestión automática de memoria mediante el recolector de basura.

Conclusión

Los conceptos avanzados de clases en C++ proporcionan un marco estructurado y potente para el desarrollo de software modular y reutilizable. La abstracción mediante clases, la composición, el uso de funciones friend, el puntero `this` y los mecanismos de gestión de memoria permiten al programador construir sistemas complejos con control y eficiencia. La comparación con Python revela diferencias sintácticas y filosóficas, pero ambos lenguajes comparten principios orientados a objetos que refuerzan la lógica del diseño de software.